Department of Mathematics
TUM School of Computation, Information and Technology
Technical University of Munich

Determinants in Integer Programming Formulations of Hard Problems

Simon Keil

Thesis for the attainment of the academic degree

Bachelor of Science

at the TUM School of Computation, Information and Technology of the Technical University of Munich

Supervisor:
Prof. Dr. rer. nat. Stefan Weltge

Advisor:
Prof. Dr. rer. nat. Stefan Weltge

Submitted:
Munich, 1. September 2023

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Munich, 1. September 2023 Simon Keil

Zusammenfassung

Eine Reihe von Resultaten der letzten Jahre [1, 2, 3, 4] lassen vermuten, dass ganzzahlige Programme mit Koeffizientenmatrizen, deren Subdeterminanten durch eine Konstante beschränkt sind, in polyomieller Zeit lösbar sein könnten. Wäre diese Vermutung wahr, so müsste (unter der Annahme P $\neq$ NP) jede Formulierung eines NP-schweren Problems als ganzzahliges Programm unbeschränkte Subdeterminanten haben. Wir stellen zunächst drei Strategien, um zu beweisen, dass unbeschränkten Subdeterminanten in einem ganzzahligen Programm vorhanden sind, vor. Wir präsentieren dann empirische Hinweise dafür, dass jede Formulierung eines NP-schweren Problems als ganzzahliges Programm unbeschränkte Subdeterminanten hat, indem wir diese Strategien auf alle üblichen Formulierungen der 21 NP-vollständigen Probleme auf der Liste von Karp [5] als ganzzahliges Programm anwenden. Darüber hinaus präsentieren wir eine neue Charakterisierung der Determiante von Matrizen mit Einträgen in $\{-1, 0, +1\}$ und genau zwei von Null verschiedenen Einträgen pro Zeile.

Abstract

A number of results in recent years [1, 2, 3, 4] suggest that integer programs that have constraint matrices whose subdeterminants are bounded by a constant might be solvable in polynomial time. If this conjecture were true, it would imply that any integer programming formulation of an NP-hard problem must necessarily feature unbounded subdeterminants (assuming P $\neq$ NP). We give empirical evidence for the truth of this consequence by formulating three strategies to prove that a given integer programming formulation of an NP-hard problem has unbounded subdeterminants and successfully applying them to all standard integer programming formulations of the 21 NP-complete problems on the list compiled by Karp [5]. Moreover, we present a novel characterization of the determinant of matrices with entries in $\{-1, 0, +1\}$ and exactly two non-zeros per row.

Contents

1 Introduction

Two of the most important tools in operations research and other fields that perform optimization for real-world problems are linear programming and integer programming. Therefore, gaining insights into these problems is highly valuable. In particular, understanding in which cases fast algorithms can and cannot be developed is of great interest.

Formally, a *linear program* is of the form

$$\max c^T x \quad \text{such that} \quad Ax \leq b$$

where $A \in \mathbb{Q}^{n \times m}$ is some rational matrix and $b \in \mathbb{Q}^m, c \in \mathbb{Q}^n$ are some rational vectors, and an *integer program* is of the form

$$\max c^T x \quad \text{such that} \quad Ax \leq b, x \in \mathbb{Z}^n \tag{1.1}$$

where $A \in \mathbb{Z}^{n \times m}$ is some integer matrix and $b \in \mathbb{Z}^m, c \in \mathbb{Z}^n$ are some integer vectors. We say that the matrix A is the constraint matrix of the integer program.

While the variant of integer programming involving maximization is the one used predominantly for practical applications, we will focus on the feasibility variant, which can be formulated as follows.

Problem (Integer programming)**.** *Given a matrix $A \in \mathbb{Z}^{n \times m}$ and a vector $b \in \mathbb{Z}^m$, decide whether there is a vector $x \in \mathbb{Z}^n$ such that $Ax \leq b$.*

This version of integer programming is an example of a *decision problem* [6]. In general, a decision problem is a set of instances combined with a question that can unambiguously be answered with either "Yes" or "No" for each instance. An instance can consist of a list of numbers, sets, or other objects like graphs, or any combination of them. For any instance, we define its *encoding length* as the number of symbols we need to write down all the information necessary to specify the instance. Clearly, the encoding length depends on the precise way that we write down these instances. Any such way is called an *encoding scheme*. However, we assume that the encoding lengths of instances under all reasonable encoding schemes are polynomially related. That is, for any two reasonable encoding schemes, there is some polynomial p such that p applied to the encoding length of any instance under the first encoding scheme is an upper bound for the encoding length of the same instance under the second one.

Having introduced these notions, we can now define the arguably most important classes of decision problems: P and NP. A decision problem is in P if there is an algorithm that correctly decides whether a given instance is a "Yes"- or "No"-instance and whose runtime is at most polynomial in the encoding length of the instance. Such an algorithm is called a *polynomial-time* algorithm. A decision problem is in NP if for any "Yes"-instance there is some certificate that proves that this instance is indeed a "Yes"-instance and an algorithm that checks the correctness of the certificate whose runtime is at most polynomial in the encoding length of the instance. We have glossed over many details here, in particular, what precisely an algorithm is, but we hope to have given some intuition for the two classes. For a formal discussion, we refer to Garey and Johnson's book [6].

From the informal definition, it is immediately apparent that integer programming is in NP: Any vector $x \in \mathbb{Z}^n$ that satisfies $Ax \leq b$ can be used as the certificate and it is easy to check in polynomial time that a given vector does indeed satisfy the inequality. In 1979 Khachiyan [7] provided the first polynomial-time algorithm for linear programming, proving that linear programming is in P.

Another important notion is that of *NP-hardness*. A decision problem A is said to be NP-hard if any problem B in NP can be reduced to this problem in polynomial time. By this, we mean that there is a mapping of instances of B to instances of A that maps "Yes"-instances to "Yes"-instances and "No"-instances to "No"-instances and which can be computed in polynomial time. We refer to a reduction of a problem to integer programming as an *integer programming formulation* of this problem.

In a seminal paper in 1971, Cook [8] showed that there are problems that are both in NP and NP-hard and coined the term *NP-complete* for these problems. In particular, this means that if one could find a polynomial time algorithm for any of these problems, one would obtain (the existence of) a polynomial time algorithm for all problems in NP. One year later, Karp [5] published a list of 21 NP-complete problems. Among these problems is integer programming, which means that integer programming is not in P unless P and NP coincide. In this sense, integer programming is a significantly more difficult problem to solve than linear programming.

However, in recent years there have been several results proving that certain classes of instances of integer programming can be solved in polynomial time. To introduce these classes, we need to establish some definitions. Let $A = (a_{i,j})_{i\in[m],j\in[n]} \in \mathbb{Z}^{m\times n}$ be an integer matrix, where we use the notation $[k] = \{1,\dots,k\}$ for any $k \in \mathbb{Z}$. For any subsets $I \subseteq [m], J \subseteq [n]$ such that $|I| = |J| > 0$, we call the determinant of the square submatrix $\tilde{A} = (a_{i,j})_{i\in I,j\in J}$ a *subdeterminant* of A. If A is the constraint matrix of an integer program, we say the determinant of $\tilde{A}$ is a subdeterminant of the integer program. Moreover, we say that an integer programming formulation of a problem *has bounded subdeterminants*, if the subdeterminants of the integer programs corresponding to all instances are bounded in absolute value by some constant. We say an integer programming formulation *has unbounded subdeterminants*, if it does not have bounded subdeterminants. Furthermore, whenever we refer to bounded subdeterminants (by some constant) we mean bounded in absolute value.

It is a well-known result that integer programs whose subdeterminants are bounded by 1 can be solved in polynomial time. This applies to both optimization and feasibility. The same was shown for integer programs whose subdeterminants are bounded by 2, first for feasibility by Veselov and Chirkov [1] and later for optimization by Artmann, Weismantel, and Zenklusen [2]. Moreover, Fiorini, Joret, Weltge, and Yudisky [3] were able to show that integer programs with constraint matrices whose subdeterminants are bounded by a constant and that have at most two non-zero entries per row can be solved in polynomial time. Lastly, a very recent advance was made by Nägele, Nöbel, Santiago, and Zenklusen [4], who were able to show that there is a randomized algorithm that runs in polynomial time to decide the feasibility of an integer program whose subdeterminants are bounded by 4.

These results might lead one to believe that while integer programming, in general, is NP-hard, integer programs whose subdeterminants are bounded by a constant can be solved in polynomial time. If this conjecture were true, it would, in particular, imply that any integer programming formulation of an NP-hard problem must necessarily feature unbounded subdeterminants (assuming P $\neq$ NP). We will give empirical evidence for the truth of this statement by examining a large number of integer programming formulations of the NP-complete problems on Karp's list [5] and proving that all of them have unbounded subdeterminants.

Note, however, that there are integer programming formulations of NP-complete problems whose constraint matrices consist of a matrix whose subdeterminants are bounded by 1 and an additional row having entries in $\{0,1\}$ [9]. Using the Laplace expansion, it is easy to see that such matrices have subdeterminants bounded by the number of columns. Therefore, for any $\varepsilon > 0$, integer programs whose subdeterminants are bounded by n^ε, where n is the number of columns of the constraint matrix, are NP-hard. This implication follows from the observation that for any matrix $A \in \mathbb{Z}^{m\times n}$ with subdeterminants bounded by n and for $k \in \mathbb{Z}_{>0}$ such that $1/k \leq \varepsilon$, the matrix $\tilde{A} = \begin{pmatrix} A & \mathbf{0} \\ \mathbf{0} & I_{n^k} \end{pmatrix} \in \mathbb{Z}^{\tilde{m}\times\tilde{n}}$, where $\mathbf{0}$ denotes the all-zero matrix of appropriate size and I_{n^k} denotes the identity matrix of size $n^k \times n^k$, has size polynomial in the size of A and has subdeterminants bounded by $\tilde{n}^\varepsilon$. This means that while we do expect to be able to find unbounded subdeterminants in integer programming formulations of hard problems, we cannot expect them to grow quickly in the number of variables in all cases.

1.1 Strategies

In order to prove that integer programming formulations have unbounded subdeterminants, we will employ three major strategies. The first one relies on the fact that there is a large number of NP-complete

problems that can be formulated in terms of graphs. More precisely, it utilizes the incidence matrix of a graph.

Definition 1.1. *Let $G = (V, E)$ be a graph. Then the incidence matrix of G is the matrix $I(G) = (i_{v,e}) \in \{0,1\}^{V \times E}$ with*

$$i_{v,e} = \begin{cases} 1, & v \in e \\ 0, & \text{otherwise} \end{cases} \qquad \text{for all } v \in V \text{ and } e \in E.$$

Many integer programming formulations of problems that are defined in terms of graphs feature the incidence matrix as a submatrix of their constraint matrix. To be able to formulate a characterization of the subdeterminants of an incidence matrix, we introduce a helpful quantity.

Definition 1.2. *Let G be a graph. Then the odd cycle packing number $\mathrm{ocp}(G)$ is defined as the maximal cardinality of a family of vertex disjoint odd cycles in G.*

In terms of the odd cycle packing number $\mathrm{ocp}(G)$ the following characterization holds.

Lemma 1.3 (Grossman, Kulkarni and Schochetman [10])**.** *For any graph G the maximal absolute value of a subdeterminant of $I(G)$ is $2^{\mathrm{ocp}(G)}$.*

Proof. This is Theorem 2.2 from [10]. Alternatively, it can be seen as a special case of Theorem 2.4, which we will establish in a subsequent section. □

Using this lemma, we are able to provide an exponential lower bound of the maximal absolute value of a subdeterminant of many integer programming formulations of graph-theoretic problems.

The second strategy is based on the fact that an integer matrix with a fixed number of columns and bounded subdeterminants can only have a limited number of distinct rows. More precisely, the following lemma holds.

Lemma 1.4. *Let $A \in \mathbb{Z}^{m \times n}$ be an integer matrix such that for all $i, j \in [m], i \neq j$, we have $A_{i,*} \notin \{\mathbf{0}, \pm A_{j,*}\}$, where $A_{i,*}$ denotes the i-th row of A and $\mathbf{0}$ denotes the all-zero vector. Moreover, let $\Delta(A) \in \mathbb{Z}_{\geq 1}$ be the maximal absolute value of a subdeterminant of A. Then*

$$\Delta(A) \geq \sqrt{\frac{2m}{n^2 + n}}$$

holds.

Proof. By adding some rows of the $n \times n$ unit matrix to A we can obtain a matrix $\tilde{A} \in \mathbb{Z}^{\tilde{m} \times n}$ such that for all $i, j \in [\tilde{m}], i \neq j$, we have $\tilde{A}_{i,*} \notin \{\mathbf{0}, \pm \tilde{A}_{j,*}\}$ and $\tilde{A}$ has an $n \times n$ submatrix whose determinant equals $\Delta(\tilde{A}) = \Delta(A)$. Theorem 2 from Lee et al. [11] applied to $\tilde{A}$ yields $m \leq \tilde{m} \leq \frac{1}{2}(n^2 + n)\Delta(A)^2$. This implies the claim. □

For example, for any integer programming formulation that features n variables and at least n^3 distinct constraints, the lemma implies that the integer programming formulation has unbounded subdeterminants.

The final strategy is based on an elementary observation concerning determinants and block matrices.

Lemma 1.5. *Let $A_i \in \mathbb{R}^{n_i \times n_i}$ be square matrices for all $i \in [n]$. Then the matrix*

$$A = \begin{pmatrix} A_1 & * & * & * \\ \mathbf{0} & A_2 & * & * \\ \vdots & \ddots & \ddots & * \\ \mathbf{0} & \cdots & \mathbf{0} & A_n \end{pmatrix},$$

where the entries marked with "$*$" are arbitrary and $\mathbf{0}$ represents the all-zero matrix of appropriate size, has determinant $\det(A) = \prod_{i=1}^{n} \det(A_i)$. We say that A is an upper block triangular matrix. *If all the entries marked with "$*$" are 0, we say that A is a* block diagonal matrix.

Proof. This follows immediately from the Leibniz formula for determinants. □

This approach is especially useful for integer programming formulations with different groups of variables and restrictions that impose a natural partition of the constraint matrix into blocks.

2 Karp's 21 NP-complete problems

In this chapter, we examine the list of 21 NP-complete problems that Karp published in 1972 [5]. We refer to the paper repeatedly but omit the explicit citation from now onward. Furthermore, the names commonly used today to refer to some problems on the list differ from those used by Karp. To be consistent with the prevailing literature, we use these differing names instead of the ones Karp originally used. Moreover, whenever an object is introduced in the definition of a problem, i. e., a graph $G = (V, E)$, we will refer to that object afterward without reintroducing it. Lastly, the order in which the problems are presented differs from the original publication because we grouped the problems thematically.

We begin the discussion with two general problems. We then move on to several graph-theoretic problems before examining some set and partition problems. We finish our investigation by looking at the problems on the list belonging to a certain subclass of NP called *weakly NP-complete*.

2.1 0-1 integer programming

Not only do we consider the problems on Karp's list with regard to integer programming, but a variant of integer programming itself is also a problem on the list.

Problem 1 (0-1 integer programming)**.** *Given a matrix $A \in \mathbb{Z}^{m \times n}$ and $b \in \mathbb{Z}^m$, decide whether there is a vector $x \in \{0, 1\}^n$ such that $Ax = b$.*

First, note that the formulation $Ax = b, x \in \{0, 1\}^n$ does not yet fit the specific form defined in (1.1). However, the constraints $Ax = b$ can be replaced by the equivalent ones $Ax \leq b, -Ax \leq -b, x \in \mathbb{Z}$, and the restriction $x \in \{0, 1\}^n$ can be replaced by $-Ix \leq 0, Ix \leq 1$, where I denotes the $n \times n$ identity matrix. Note that this altered formulation does fit (1.1), and that the maximal absolute values of the subdeterminants of both formulations are equal. Since an analogous transformation can be performed for any integer program with equality constraints and bounds on variables, we will omit this step going forward.

Since there are no restrictions placed on the choice of the matrix $A \in \mathbb{Z}^{m \times n}$, it is obvious that there are instances that feature arbitrarily large subdeterminants. Moreover, in the subsequent problems, we will see other problems that can be reduced to restricted families of instances of 0-1 integer programming and yield unbounded subdeterminants nonetheless.

2.2 Satisfiability

The first problem proven to be NP-complete in the seminal paper by Cook [8] was the satisfiability problem. Informally the problem is the following: Given a set of boolean variables $\{x_1, \ldots, x_n\}$, i. e. variables that can be set to either *true* or *false*, and some boolean formula involving $\wedge$ (conjunction), $\vee$ (disjunction), $\overline{(\cdot)}$ (negation) and parentheses, decide whether there is some assignment of the boolean variables such that the formula is true.

The expression

$$(x_1 \vee x_3 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_4) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee x_3 \vee x_4) \tag{2.1}$$

is an example of such a formula. Actually, this example is of a special form as it is a conjunction of multiple smaller formulae that are all disjunctions of one or more variables or their negation. Boolean formulae that follow this pattern are said to be in *conjunctive normal form*, and any boolean formula using the symbols described above can be transformed into an equivalent one in conjunctive normal form [12]. Therefore,

it is sufficient to only consider formulae in conjunctive normal form, simplifying the formal introduction significantly.

The phrasing of the following formal definition is adapted from Garey and Johnson [6].

Definition 2.1. *Let $X = \{x_1, \dots, x_n\}$ be a set of boolean variables. Any function $t : X \to \{\text{true}, \text{false}\}$ is called a* truth assignment *for X. For every variable $x_i, i \in [n]$, we say that x_i and its negation $\overline{x}_i$ are literals over X. We extend any truth assignment t to all literals by defining $t(\overline{x}_i) = \text{true}$ if $t(x_i) = \text{false}$ and $t(\overline{x}_i) = \text{false}$ if $t(x_i) = \text{true}$. A* clause *c over X is a set of literals over X. We call a truth assignment t satisfying for a collection of clauses C if for any clause $c \in C$ at least one literal in c is true under t.*

For example, the formula from (2.1) corresponds to the set of clauses

$$C = \{\{x_1, x_3, \overline{x}_2\}, \{\overline{x}_1, \overline{x}_4\}, \{\overline{x}_1, \overline{x}_2, x_3, x_4\}\},$$

and has the satisfying truth assignment $t : x_1 \mapsto \text{false}, x_i \mapsto \text{true}$ for $i \in \{2, 3, 4\}$.

In the remainder of this section, we will use Latin characters to distinguish literals from the two sets $\{x_1, \dots, x_n\}$ and $\{\overline{x}_1, \dots, \overline{x}_n\}$, that is, a literal x will always satisfy $x \in \{x_1, \dots, x_n\}$, a literal $\overline{x}$ will always satisfy $\overline{x} \in \{\overline{x}_1, \dots, \overline{x}_n\}$. On the other hand, we will use Greek characters if the distinction is not necessary. That is, a literal χ can be in either $\{x_1, \dots, x_n\}$ or $\{\overline{x}_1, \dots, \overline{x}_n\}$. Lastly, if $\chi = \overline{x}_i$ for some $i \in [n]$, we define $\overline{\chi} = x_i$.

We are now ready to formally introduce the satisfiability problem.

Problem 2 (Satisfiability)**.** *Given a set of boolean variables $X = \{x_1, \dots, x_n\}$ and a finite set of clauses C over X, decide whether there is a satisfying truth assignment.*

Karp showed that the problem remains NP-complete when restricting the clauses to have only 3 literals. This problem is referred to as 3-SAT. More generally, we define the k-SAT problem.

Problem 3 (k-SAT)**.** *Given a set of boolean variables $X = \{x_1, \dots, x_n\}$, a finite set of clauses C over X and an integer $k \in \mathbb{N}$ such that $|c| = k$ for all $c \in C$, decide whether there is a satisfying truth assignment.*

The straightforward integer programming formulation for this problem is

$$\begin{aligned} \sum_{i \in [n]\colon x_i \in c} t_i + \sum_{i \in [n]\colon \overline{x}_i \in c} (1 - t_i) &\geq 1 && \text{for all } c \in C, \\ t_i &\in \{0, 1\} && \text{for all } i \in [n]. \end{aligned}$$

To examine subdeterminants, we first consider the more restrictive setting of 2-SAT. Note that we can restrict ourselves to those instances where the variables corresponding to the literals that appear in any one clause are distinct, i. e. there is no literal χ and clause $c \in C$ such that $\chi, \overline{\chi} \in c$. In this case, there is a one-to-one correspondence of constraint matrices of instances of 2-SAT and matrices with entries in $\{-1, 0, 1\}$ and exactly two non-zeros per row. To quantify the subdeterminants of these matrices, we introduce some helpful notions, one of which is the implication graph used by Aspvall, Plass, and Tarjan [13] as the basis of a linear-time algorithm for 2-SAT.

Definition 2.2 (Implication Graph)**.** *Let A be the constraint matrix of an instance of 2-SAT with clauses C and literals V. Set $E = \{(\overline{\chi}, \xi) : \{\chi, \xi\} \in C\}$. Then the digraph $D_A = (V, E)$ is called the* implication graph of A.

D_A is the graph that encodes the implications that can be derived from the clauses, hence the name. For instance, the clause $\{\chi, \xi\}$ representing $(\chi \vee \xi)$ leads to the implications $\overline{\chi} \Rightarrow \xi$ and $\overline{\xi} \Rightarrow \chi$, encoded by the edges $(\overline{\chi}, \xi)$ and $(\overline{\xi}, \chi)$. This motivates the notation $\chi \Rightarrow \xi$ for an edge $(\chi, \xi) \in E(D_A)$, which we will use going forward. Furthermore, we see that for $(\chi \Rightarrow \xi) \in E(D_A)$ the contrapositive $(\overline{\xi} \Rightarrow \overline{\chi})$ is also in $E(D_A)$. We link these pairs by defining $\operatorname{cp}(\chi \Rightarrow \xi) = (\overline{\xi} \Rightarrow \overline{\chi})$.

Definition 2.3. *Let A be the constraint matrix of an instance of 2-SAT, D_A be the corresponding implication graph and let $W = \chi_1, e_1, \chi_2, \dots, \chi_k, e_k, \chi_{k+1}$ be a sequence of vertices and edges in D_A. We say W is an* undirected walk *in D_A if $e_i \in \{(\chi_i \Rightarrow \chi_{i+1}), (\chi_{i+1} \Rightarrow \chi_i)\}$ for $i \in [k]$. We say W is an* undirected cycle *in D_A if W is an undirected walk, $\chi_i \neq \chi_j$ for all $i, j \in [k]$ with $i \neq j$, and $\chi_1 = \chi_{k+1}$. We say W is* contrapositive-symmetric *if for every vertex χ in W, the corresponding vertex $\overline{\chi}$ is also in W and for every edge e in W the corresponding edge $\mathrm{cp}(e)$ is also in W.*

In Figure 2.1 the constraint matrix A and the implication graph D_A of the instance

$$(\overline{x}_1 \vee x_3) \wedge (x_1 \vee \overline{x}_2) \wedge (x_2 \vee x_3) \wedge (x_1 \vee x_4) \wedge (\overline{x}_2 \vee \overline{x}_4) \tag{2.2}$$

are depicted. In D_A, we can find the two undirected walks

$$W_1 : x_1 \Rightarrow x_3 \Leftarrow \overline{x}_2 \Leftarrow \overline{x}_1 \qquad \text{and} \qquad W_2 : \overline{x}_1 \Leftarrow \overline{x}_3 \Rightarrow x_2 \Rightarrow x_1.$$

We omit a formal definition of the notation used here since it is a natural extension of the previously introduced notation for edges of D_A. By joining W_1 and W_2, we obtain an undirected cycle W in D_A. By the partition into W_1 and W_2, it is obvious that W is contrapositive-symmetric. It is straightforward to see that we can decompose any contrapositive-symmetric undirected cycle into two walks in this way.

$$A = \begin{pmatrix} -1 & 0 & 1 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & -1 & 0 & -1 \end{pmatrix}, \quad D_A\text{:}$$

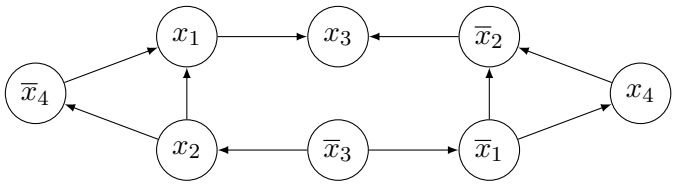

Figure 2.1 Constraint matrix A and implication graph D_A corresponding to the instance (2.2) of 2-SAT.

By exhaustive search, one can verify that the maximal determinant of a square submatrix of A is 2, attained by the top-left 3×3 submatrix that corresponds to W. This is not a coincidence, as the following theorem shows.

Theorem 2.4. *Let $A \in \{-1, 0, 1\}^{N \times M}$ be a matrix with exactly two non-zero entries per row and let k be the maximal cardinality of a family of vertex disjoint, contrapositive-symmetric undirected cycles in D_A. Then the maximal absolute value of a subdeterminant of A is 2^k.*

Proof. Let $B \in \{-1, 0, 1\}^{n \times n}$ be any square submatrix of A. If B has a row or column with all entries equal to 0, the determinant is 0, hence it is not maximal. If B has a row or column with only one non-zero entry, expansion along this row or column yields a smaller matrix B' with $|\det(B)| = |\det(B')|$. By induction, we can assume B to have at least two non-zeros per row and column. Since A has only two non-zero entries per row, B has exactly two non-zero entries per row and column. By permutation of rows and columns and multiplying some rows by -1, we can transform B into a block diagonal matrix

$$\begin{pmatrix} B_1 & 0 & \cdots & 0 \\ 0 & B_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & B_l \end{pmatrix}$$

where each block $B_i \in \{-1,0,1\}^{n_i \times n_i}$ is of the form

$$B_i = \begin{pmatrix} -1 & 0 & \cdots & \cdots & \cdots & 0 & 1 \\ 1 & \ddots & \ddots & & & \vdots & 0 \\ 0 & \ddots & -1 & \ddots & & \vdots & \vdots \\ \vdots & \ddots & 1 & 1 & \ddots & \vdots & \vdots \\ \vdots & & \ddots & 1 & \ddots & 0 & \vdots \\ \vdots & & & \ddots & \ddots & 1 & 0 \\ \underbrace{0 \quad \cdots \quad \cdots}_{m_i} & & & \underbrace{\cdots \quad 0 \quad 1 \quad 1}_{n_i - m_i} & & & \end{pmatrix}.$$

that is $(B_i)_{j+1,j} = 1$ for $j \in [n-1]$, $(B_i)_{j,j} = -1$ for $j \in [m_i]$, $(B_i)_{j,j} = 1$ for $j \in \{m_i+1, \dots, n_i\}$, and $(B_i)_{1,n_i} = 1$. Note that none of these operations change the absolute value of the determinant of B. Furthermore, permutations correspond to a renumbering of the variables and clauses. Finally, multiplying a row by -1 corresponds to reversing an edge e and the corresponding edge $\mathrm{cp}(e)$.

If $m_i = n_i$ the sum of all columns of B_i is $\mathbf{0} \in \mathbb{Z}^{n_i}$, implying the determinant is 0. Otherwise, expansion along the last row (and subsequent expansion along the last column for the first summand) yields

$$\det(B_i) = (-1)(-1)^{n_i} + (-1)^{m_i} = \begin{cases} +2, & \text{if } m_i \text{ even, } n_i \text{ odd} \\ -2, & \text{if } m_i \text{ odd, } n_i \text{ even} \\ 0, & \text{otherwise.} \end{cases}$$

We now show the correspondence of a block B_i with a non-zero determinant to an undirected, contrapositive-symmetric cycle C_i in D_A and vice versa. First, note that every column of A corresponds to two vertices y and $\overline{y}$ of D_A. Write $y_j, \overline{y}_j \in V(D_A)$ for the vertices corresponding to the j-th column of B_i and set $y_0 = y_{n_i}$. The l-th row of B_i yields the implications $(y_l \Rightarrow y_{l-1}), (\overline{y}_{l-1} \Rightarrow \overline{y}_l) \in E(D_A)$ for $l \in [m_i]$ and the implications $(\overline{y}_l \Rightarrow y_{l-1}), (\overline{y}_{l-1} \Rightarrow y_l) \in E(D_A)$ for $l \in \{m_i+1, \dots, n_i\}$. This means B_i corresponds to the undirected walks

$$y_{n_i} \Leftarrow y_1 \Leftarrow y_2 \Leftarrow y_3 \dots y_{m_i} \Leftarrow \overline{y}_{m_i+1} \Rightarrow y_{m_i+2} \Leftarrow \overline{y}_{m_i+3} \Rightarrow y_{m_i+4} \dots \begin{cases} \Leftarrow \overline{y}_{n_i}, \text{ if } n_i - m_i \text{ is odd} \\ \Rightarrow y_{n_i}, \text{otherwise.} \end{cases}$$

$$\overline{y}_{n_i} \Rightarrow \overline{y}_1 \Rightarrow \overline{y}_2 \Rightarrow \overline{y}_3 \dots \overline{y}_{m_i} \Rightarrow y_{m_i+1} \Leftarrow \overline{y}_{m_i+2} \Rightarrow y_{m_i+3} \Leftarrow \overline{y}_{m_i+4} \dots \begin{cases} \Rightarrow y_{n_i}, \text{ if } n_i - m_i \text{ is odd} \\ \Leftarrow \overline{y}_{n_i}, \text{otherwise.} \end{cases}$$

in D_A. If $n_i - m_i$ is odd, the two walks link to form a contrapositive-symmetric, undirected cycle. If $n_i - m_i$ is even, the two walks are each an undirected cycle that is not contrapositive-symmetric.

For the converse direction, let C be an undirected, contrapositive-symmetric cycle in D_A. Let χ_1 be a vertex in C. Since C is contrapositive-symmetric, $\overline{\chi}_1$ is also in C. This means there is a walk $W = \chi_1, e_1, \dots, \chi_n, e_n, \overline{\chi}_1$ contained in C. Leveraging contrapositive symmetry again, we see that C is obtained by joining W and $\mathrm{cp}(W) := \overline{\chi}_1, \mathrm{cp}(e_1), \dots, \overline{\chi}_n, \mathrm{cp}(e_n), \chi_1$. Let B_C be the submatrix of A that is obtained by selecting the rows corresponding to $e_1, \dots, e_n$ and selecting the columns corresponding to $\chi_1, \dots, \chi_n$. Since C is a cycle and by the partition of C into W and $\mathrm{cp}(W)$, we see that B_C is a square matrix with exactly two non-zero entries per row. Let $R \subseteq [n]$ be the rows of B which have exactly one $+1$ and one -1 entry. Observe that any edge in W corresponding to a row $i \in R$ connects two vertices $x, y \in V(D_A)$ or two vertices $\overline{x}, \overline{y} \in V(D_A)$, whereas edges corresponding to rows $i \in [n] \setminus R$ connect two vertices $x, \overline{y} \in V(D_A)$. Since W starts in χ_1 and ends in $\overline{\chi}_1$ this means that $n - |R|$ has to be odd. However, this means that after some permutation of the rows and columns and multiplication of some columns by -1, B_C is of the claimed form.

Finally, let $C_1, \dots, C_l$ be undirected, contrapositive-symmetric cycles in D_A and let $B_1, \cdots, B_l$ be the corresponding submatrices of A. Since the cycles are contrapositive-symmetric, the columns of A corresponding to the columns of the submatrices are disjoint if and only if the cycles are vertex disjoint.

Therefore, the submatrix obtained by selecting all the rows and columns selected in $B_1, \cdots, B_l$ is a block diagonal matrix if and only if the cycles $C_1, \ldots, C_l$ in D_A are vertex disjoint. □

Given an instance of 2-SAT, we can construct an equivalent instance of 3-SAT by adding a new variable corresponding to the literals $x^*, \overline{x}^*$ and replacing any clause $\{\chi, \xi\}$ by the two clauses $\{\chi, \xi, x^*\}$ and $\{\chi, \xi, \overline{x}^*\}$. Clearly, the maximal absolute value of a subdeterminant of the constraint matrix of the 2-SAT instance is a lower bound for that of the corresponding 3-SAT instance. Combining this with Theorem 2.4 yields a way to construct instances of 3-SAT that feature arbitrarily large subdeterminants.

A characterization similar to Theorem 2.4 of the maximal absolute value of a subdeterminant of arbitrary instances of 3-SAT would be desirable. However, the presence of three non-zero entries renders the approach taken for 2-SAT impossible. In any case, such a characterization would only allow us to understand more precisely for which instances unbounded subdeterminants appear, as the approach via 2-SAT already yields the existence of such instances.

Another way to produce instances that feature unbounded subdeterminants is to consider the instances whose constraint matrices are incidence matrices of some graphs with an appended column of ones. Lemma 1.3 implies that there are instances of this type that lead to unbounded subdeterminants. However, all of these instances are trivially solvable. In contrast, the instances derived from 2-SAT are not trivially solvable, albeit in polynomial time. Unfortunately, this means that we cannot answer whether there are instances for which no polynomial-time algorithm is known and that feature unbounded subdeterminants. Nevertheless, Theorem 2.4 provides strong evidence in favor of the existence of such instances.

2.3 Independent set and clique problems

In this section, we will consider the first four problems phrased in terms of graphs. Recall that for a graph $G = (V, E)$, a subset $S \subseteq V$ is called a *clique* of G if any two distinct vertices in S are adjacent and that it is called an *independent set* of G if no two vertices in S are adjacent. Furthermore, we say $S \subseteq V$ is a *vertex cover* of G if any edge $e \in E$ is incident to at least one vertex in S.

Writing $\overline{G} = (V, \overline{E})$, where $\overline{E} = \{\{v, w\} : v, w \in V, \{v, w\} \notin E\}$, for the *complement graph* of G, we easily see the following equivalences

$$S \text{ is a clique of } \overline{G} \quad \Leftrightarrow \quad S \text{ is an independent set of } G \quad \Leftrightarrow \quad V \setminus S \text{ is a vertex cover of } G. \tag{2.3}$$

Equipped with these definitions and observations, we can now formally define the problems themselves.

2.3.1 Vertex cover and clique

The first problem deals with a vertex cover of a certain size.

Problem 4 (Vertex cover)**.** *Given a graph $G = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a vertex cover $S \subseteq V$ of G such that $|S| \leq k$.*

The second one asks for a clique of a certain size.

Problem 5 (Clique)**.** *Given a graph $G = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a clique $S \subseteq V$ of G such that $|S| \geq k$.*

By the equivalence established in (2.3), any instance of the vertex cover problem can be seen as an instance of the clique problem by replacing G with $\overline{G}$ and k by $n - k$, and vice versa. For this reason, any integer programming formulation for one of the problems immediately yields a formulation for the other. Therefore, only considering integer programming formulations for the clique problem suffices.

Kardos, Patassy, Szabo, and Zavalnij [14] present four integer programming formulations for the clique problem. All of them use binary variables $x_v \in \{0, 1\}$ for all $v \in V$ that indicate which vertices are selected as part of the clique, and the restriction $\sum_{v \in V} x_v \geq k$ is imposed to ensure the clique is sufficiently large.

The first formulation is called the *edge reformulation* and completes this approach by adding the restrictions

$$x_v + x_w \leq 1 \qquad \text{for all } \{v, w\} \in \overline{E}.$$

Since the constraint matrix of these restrictions is the incidence matrix of $\overline{G}$, Lemma 1.3 implies that this formulation features unbounded subdeterminants.

The second formulation they propose is the *independent set formulation*. This formulation appends the constraints

$$\sum_{v \in S} x_v \leq 1 \qquad \text{for each independent set } S \text{ of } G. \tag{2.4}$$

Note, however, that some of these constraints are redundant. In particular, any restriction in (2.4) corresponding to an independent set that is a proper subset of another independent set of G can be omitted without change to the feasible region of the formulation. Any independent set of G that is not a proper subset of another independent set is called a *maximal independent set*. However, there are graphs with $3^{|V|/3}$ many maximal independent sets (namely the disjoint union of $|V|/3$ triangles) [15]. This means that even if only maximal independent sets are considered, there can still be exponentially many distinct constraints in (2.4). Lemma 1.4 therefore implies that there are instances that lead to unbounded subdeterminants for this formulation as well.

The third formulation utilizes the set of non-neighbors $N_v = \{w \in V \setminus \{v\} : \{v, w\} \notin E\}$ for all $v \in V$. Based on these, they define the coefficients $a_v = |N_v|$, and add the restrictions

$$a_v x_v + \sum_{w \in N_v} x_w \leq a_v \qquad \text{for all } v \in V. \tag{2.5}$$

Observe that unless the graph is very dense, that is, almost all possible edges are present, there will be coefficients a_v that are relatively large. Let $\varepsilon > 0$ be a constant. Then, in particular, any family of graphs that have vertices which are adjacent to at most $(1 - \varepsilon)|V|$ vertices, yields a family of instances of the clique problem whose constraint matrices have subdeterminant that grow linearly in $|V|$, where V is the set of vertices of the respective instances.

The final formulation Kardos et al. present is a modified version of the previous one. They fix an arbitrary ordering of the vertices and modify the sets N_v to only include vertices that succeed v in this ordering. The constraints (2.5) are adapted accordingly. Still, this formulation has unbounded subdeterminants for the same reason as the previous formulation.

2.3.2 Chromatic number and clique cover

The next two problems are concerned with partitions of a graph into subgraphs with certain properties. To introduce the first one, we define a *graph coloring using k colors* of a graph $G = (V, E)$ as a function $\phi : V \to [k]$ such that $\phi(v) \neq \phi(w)$ for all $\{v, w\} \in E$. We say that G is *k-colorable* if there is a graph coloring of G using k colors.

Problem 6 (Chromatic number)**.** *Given a graph $G = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether G is k-colorable.*

The name of the problem stems from the fact that the minimal $k \in \mathbb{Z}_{>0}$ such that G is k-colorable is referred to as the *chromatic number* of G. Therefore the problem can equivalently be phrased as deciding whether the chromatic number of G is at most k.

By the definition of a coloring, it is clear that all vertices assigned to the same color form an independent set. Complementary, a partition of the vertices of G into independent sets yields a coloring. This means another equivalent formulation is to ask whether a partition of the vertices of G into at most k independent sets exists. This leads to another similar problem.

Problem 7 (Clique cover). *Given a graph $G = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a partition of V into at most k cliques.*

Combining the observation made before with (2.3) implies that any instance of the chromatic number problem can be seen as an instance of the clique cover problem by replacing G with $\overline{G}$ and vice versa, just like it was the case with the vertex cover problem and the clique problem before. Therefore, we will only consider integer programming formulations for the chromatic number problem, as they immediately yield corresponding formulations for the clique cover problem.

The classical integer programming formulation of the chromatic number problem is

$$\begin{aligned} \sum_{i=1}^{k} x_{v,i} &= 1 && \text{for all } v \in V \text{ and all } i \in [k], \\ x_{v,i} + x_{w,i} &\leq 1 && \text{for all } \{v, w\} \in E \text{ and all } i \in [k], \\ x_{v,i} &\in \{0, 1\} && \text{for all } v \in V \text{ and all } i \in [k], \end{aligned}$$

where the variable $x_{v,i}$ indicates whether the vertex v is assigned the color i for all $v \in V$ and $i \in [k]$ [16]. Evidently, the incidence matrix of G is a submatrix of the constraint matrix of this formulation. Again, Lemma 1.3 implies that there are instances that have arbitrarily large subdeterminants. Based on this classical formulation, a new formulation was brought forward by Méndez-Díaz and Zabala [16], which improves the formulation in some regards. Nonetheless, this altered formulation still features the incidence matrix of G as a submatrix.

Furthermore, a formulation similar to the independent set formulation (2.4) for the clique problem was introduced by Mehrotra and Trick [17]. Writing $\mathcal{I} = \{I \subseteq V : I \text{ is a maximal independent set of } G\}$, this formulation introduces variables $x_I \in \{0, 1\}$ for all maximal independent sets $I \in \mathcal{I}$ and enforces the constraints

$$\begin{aligned} \sum_{I \in \mathcal{I}} x_I &\leq k, \\ \sum_{I \in \mathcal{I}:\, v \in I} x_I &\geq 1 && \text{for all } v \in V. \end{aligned}$$

By the argument made when discussing (2.4), the constraint matrix of this formulation can have exponentially many columns in $|V|$. Applying Lemma 1.4 to the transpose matrix implies that this formulation has unbounded subdeterminants.

Yet another formulation, based on the idea to choose a representative vertex for each color, was proposed by Campêlo, Corrêa, and Frota [18] and later improved by Campêlo, Campos, and Corrêa [19]. However, these two formulations contain the incidence matrix of G as a submatrix of the constraint matrix as well. Lastly, further formulations based on partial orderings which have unbounded subdeterminants for the same reason were presented by Jabrayilov and Mutzel [20, 21].

2.4 Feedback vertex set and feedback arc set

Similar to the previous section, we will examine two complementary problems jointly in this section. The feedback vertex set problem asks for the deletion of a certain number of vertices and incident edges to leave the graph acyclic. The feedback arc set problem asks whether a given digraph can be turned into an acyclic one by deleting a certain number of edges. Formally, they can be stated as presented below.

Problem 8 (Feedback vertex set). *Given a digraph $D = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a set $S \subseteq V$ of at most k vertices, such that any directed cycle in D contains a vertex in S.*

Problem 9 (Feedback arc set). *Given a digraph $D = (V, E)$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a set $S \subseteq E$ of at most k edges such that any directed cycle in D contains an edge in S.*

For both problems, there is a straightforward integer program formulation that Chudak, Goemans, Hochbaum, and Williamson [22] call the *standard cycle formulation*:

$$\begin{aligned} \sum_{i \in X} x_i &\leq k, \\ \sum_{i \in C} x_i &\geq 1 && \text{for all } C \in \mathcal{C}_X, \\ x_i &\in \{0,1\} && \text{for all } i \in X, \end{aligned}$$

with $X = V$ and $X = E$ respectively, and where $\mathcal{C}_V$, respectively $\mathcal{C}_E$, denotes the set of all vertex sets, respectively edge sets, of all cycles of D.

We now consider the reductions of an instance of the vertex cover problem to these problems proposed by Karp. Let $G = (V', E')$ together with an integer $l \in \mathbb{Z}_{>0}$ be an instance of the vertex cover problem. Then the proposed reductions are

$$D = (V', E),\ k = l, \quad \text{where } E = \{(v, w) : \{v, w\} \in E'\}$$

for feedback vertex set and

$$D = (V' \times \{0,1\}, E),\ k = l, \quad \text{where } E = \{((u,0),(u,1)) : u \in V'\} \cup \{((u,1),(w,0)) : \{v,w\} \in E'\}$$

for feedback arc set. It is easy to verify that the constraint matrices of the standard cycle formulation of these reductions contain the incidence matrix of G as a submatrix. This means they can have arbitrarily large subdeterminants.

Chudak et al. [22] proposed a new integer programming formulation for the feedback vertex set problem as an alternative to the standard cycle formulation. However, this formulation features a restriction for every $S \subseteq V$ such that $E \cap (S \times S) \neq \emptyset$, which in combination with Lemma 1.4 yields the presence of unbounded subdeterminants in this formulation as well.

Using the fact that a digraph is acyclic if and only if there is a topological ordering, Baharev, Schichl, Neumaier, and Achterberg [23] presented an alternative integer programming formulation of the feedback arc set problem based on an integer programming formulation for an ordering by Grötschel, Jünger, and Reinelt [24]. It asks for an ordering of the vertices, such that fewer than k edges "violate" this ordering. To define the formulation, we write $V = \{v_1, \ldots, v_n\}$ such that $|V| = n$. The formulation introduces variables $y_{i,j} \in \{0,1\}$ for all $1 \leq i < j \leq n$ which take the value 0 if i precedes j in the ordering and 1 otherwise. Formally, it reads

$$\begin{aligned} \sum_{1 \leq i < j \leq n : (v_i, v_j) \in E} y_{i,j} + \sum_{1 \leq i < j \leq n : (v_j, v_i) \in E} (1 - y_{i,j}) &\leq k, \\ y_{i,j} + y_{j,k} - y_{i,k} &\leq 1 && \text{for all } 1 \leq i < j < k \leq n, \\ -y_{i,j} - y_{j,k} + y_{i,k} &\leq 0 && \text{for all } 1 \leq i < j < k \leq n, \\ y_{i,j} &\in \{0,1\} && \text{for all } 1 \leq i < j \leq n. \end{aligned} \tag{2.6}$$

To see that this formulation is not spared from unbounded subdeterminants, consider the submatrix A induced by selecting the columns corresponding to

$$y_{i,i+1}, \quad y_{i+1,i+2}, \quad y_{i+2,i+3}, \quad y_{i+3,i+4}, \quad y_{i,i+4}$$

and the rows corresponding to

$$y_{i,j} + y_{j,k} - y_{i,k} \leq 1$$

for $(i,j,k) \in \{(i,i+1,i+2),(i+1,i+2,i+3),(i+2,i+3,i+4),(i,i+3,i+4),(i,i+1,i+4)\}$ and for all $i \in \{5k : k \in \mathbb{Z}_{>0}, 5k+4 \leq n\}$. Note that the submatrix $\tilde{A}$ corresponding to a fixed i is given by

$$\tilde{A} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 1 & 0 & 0 & 0 & -1 \end{pmatrix},$$

if the rows and columns are ordered accordingly and satisfies $\det(\tilde{A}) = -2$. Furthermore, observe that after suitable permutation of the rows and columns, A forms a block diagonal matrix with $\lfloor n/5 \rfloor$ copies of $\tilde{A}$ on the diagonal, which means $|\det(A)| = 2^{\lfloor n/5 \rfloor}$ yields an exponential lower bound on the maximal absolute value of a subdeterminant of the constraint matrix of this integer program.

Similar arguments will be used again in subsequent sections. In these arguments, we will implicitly assume a suitable permutation of the rows and columns without explicitly mentioning it.

2.5 Hamiltonian cycle

In this section, we will consider the Hamiltonian cycle problem, which asks whether there is a cycle in a graph that visits all vertices.

Problem 10 (Hamiltonian cycle)**.** *Given a graph $G = (V,E)$, decide whether it contains a Hamiltonian Cycle, i. e. an ordering $f : [|V|] \to V$ such that $\{f(i), f(i+1)\} \in E$ for all $i \in [|V|-1]$ and $\{f(|V|), f(1)\} \in E$.*

For the sake of completeness, we will also introduce the directed version of the problem, which also holds a spot on Karp's list.

Problem 11 (Directed Hamiltonian cycle)**.** *Given a digraph $D = (V,E)$, decide whether it contains a directed Hamiltonian Cycle, i. e. an ordering $f : [|V|] \to V$ such that $(f(i), f(i+1)) \in E$ for all $i \in [|V|-1]$ and $(f(|V|), f(1)) \in E$.*

However, we will only examine the undirected version. The analysis of the directed version can be accomplished completely analogously and offers little additional insight.

These problems are closely related to the traveling salesperson problem, which asks for the length of a shortest Hamiltonian cycle in a complete graph with given edge lengths [6]. For the traveling salesperson problem, a plethora of integer programming formulations exist, the earliest dating back to at least 1954 [25]. These formulations can be adapted in a straightforward manner to yield integer programming formulations for the Hamiltonian cycle problem. Some of these adapted formulations will be presented in the remainder of the section.

All formulations which we will consider fall into one of two categories: The first one has variables $x_e \in \{0,1\}$ for every undirected edge $e \in E$, and the restrictions

$$\sum_{w:\{v,w\}\in E} x_{\{v,w\}} = 2 \qquad \text{for all } v \in V, \tag{2.7}$$

the second one has variables $x_{(v,w)} \in \{0,1\}$ for the two directed versions of every edge $\{v,w\} \in E$ and the restrictions

$$\begin{aligned} \sum_{v:\{v,w\}\in E} x_{(v,w)} &= 1 \qquad \text{for all } w \in V \\ \sum_{w:\{v,w\}\in E} x_{(v,w)} &= 1 \qquad \text{for all } v \in V. \end{aligned} \tag{2.8}$$

In both cases, the restrictions ensure that the set of edges selected by the variables that take the value 1 in a solution form a set of cycles. To complete the formulations, restrictions that ensure that only one cycle is present in a solution are added.

2.5.1 Subtour elimination

The first formulation of such additional constraints is due to Dantzig, Fulkerson, and Johnson [25]. It uses (2.7) and adds one of the following two families of constraints

$$\sum_{\{v,w\}\in E:\, v\in S,\, w\notin S} x_{\{v,w\}} \geq 2 \qquad \text{for all } S \subsetneq V \text{ with } \emptyset \neq S, \tag{2.9}$$

or

$$\sum_{\{v,w\}\in E:\, v\in S, w\in S} x_{\{v,w\}} \leq |S| - 1 \qquad \text{for all } S \subsetneq V \text{ with } \emptyset \neq S. \tag{2.10}$$

These families of constraints are called *subtour elimination contraints*.

Notice that the constraint matrix of the inequalities given by (2.7) is the incidence matrix of the graph G. However, the Hamiltonian cycle problem remains NP-hard even when only considering bipartite graphs [26] whose incidence matrices have subdeterminants bounded by 1. To find large subdeterminants in this case, we take a closer look at the restrictions in (2.9) and (2.10).

If G is connected, two nonempty sets $S, S' \subsetneq [n]$ yield the same restriction in (2.9) only if $S = S'$ or $S = V \setminus S'$. To see this, let $S, S' \subsetneq V$ be two nonempty subsets corresponding to the same restriction. Then $S \cap S' \neq \emptyset$ or $S \cap (V \setminus S') \neq \emptyset$ holds. Without loss of generality, let $S \cap S' \neq \emptyset$ and let $v \in S \cap S'$. Since S and S' correspond to the same restriction, for all neigbors $w \in V$ of v either $w \in S \cap S'$ or $w \in (V \setminus S) \cap (V \setminus S')$ holds. Since the graph is connected and by induction on the minimal length of a path connecting v and w, this is true for any $w \in V$, which implies $S = S'$, completing the argument. This means that there are $2^{|V|-1} - 1$ distinct restrictions in the family (2.9). Lemma 1.4 then implies that the maximal absolute value of a subdeterminant of the constraint matrix of (2.9) grows at least exponentially in $|V|$.

Clearly, the number of constraints in (2.10) is also exponential. However, unlike in the case of (2.9), there can be many duplicates among them. To see this, assume that G has an independent set S of size $k > 1$. Then all 2^k subsets of S give the same restrictions in (2.10). Note, however, that to obtain a proof of the presence of unbounded subdeterminants via Lemma 1.4, it suffices to find a relatively small number of distinct restrictions, say proportional to $|E|^3$. To this end, consider $\mathcal{S} = \{e_1 \cup e_2 \cup e_3 : e_1, e_2, e_3 \in E, |\{e_1, e_2, e_3\}| = 3\}$, that is all sets of vertices that appear as the endpoints of a set of exactly three edges. Clearly, any element $S \in \mathcal{S}$ corresponds to a unique restriction in (2.10). Furthermore, two distinct sets of three edges corresponding to the same set $S \in \mathcal{S}$ can be viewed as two distinct subgraphs of the complete graph on $|S|$ vertices, as illustrated in Figure 2.2. Since there is only a constant number of such subgraphs, only a constant number of sets of three edges can yield the same set $S \in \mathcal{S}$, which means $|\mathcal{S}|$ is at most a constant factor smaller than $\binom{|E|}{3}$. Applying Lemma 1.4 now yields subdeterminants with absolute value at most a constant factor smaller than $\sqrt{|E|}$ for large $|E|$. It is easy to see that similar arguments can be used to generate even larger lower bounds on the maximal absolute value of a subdeterminant of the constraint matrix of (2.10).

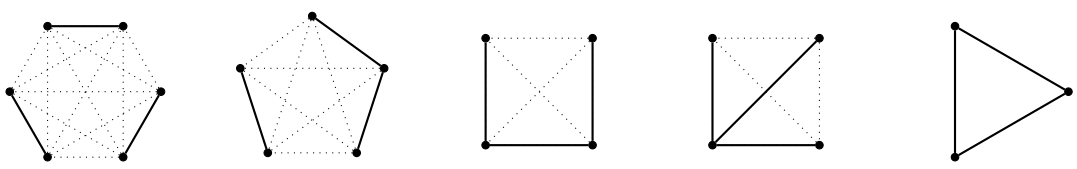

Figure 2.2 All possible configurations (up to isomorphisms) of three distinct edges as subgraphs of the complete graph on the respective number of vertices.

2.5.2 Miller-Tucker-Zemlin

A more compact formulation was proposed by Miller, Tucker, and Zemlin [27]. For this formulation, choose any $v^* \in V$ and augment (2.7) by the following set of additional constraints

$$\begin{aligned} y_v - y_w + |V|x_{(v,w)} &\leq |V| - 1 && \text{for all } \{v,w\} \in E \text{ such that } v, w \neq v^* \\ y_v &\in \mathbb{Z} && \text{for all } v \in V. \end{aligned} \tag{2.11}$$

The additional variables impose an ordering of the vertices within the tour.

Since $|V|$ appears as a coefficient, the largest subdeterminant of the constraint matrix clearly grows at least linearly in $|V|$. However, we even see that for any variable $x_{(v,w)}$ with $v, w \neq v^*$, there is exactly one inequality in the family (2.11) which has coefficient $|V|$ for this variable and in all other inequalities of the family this variable has coefficient 0. There are $|E| - \deg(v^*)$ many edges that are not incident to v^*, therefore there are $2(|E| - \deg(v^*))$ such variables, where $\deg(v)$ denotes the number of vertices adjacent to v in G. This means that the corresponding matrix has determinant $|V|^{2(|E|-\deg(v^*))}$. For any reasonable instance, this leads to an exponential lower bound on the largest subdeterminant.

2.5.3 Flow based formulations

We close the section by looking at some formulations whose underlying idea is to use some kind of flow to arrive at a completed formulation starting from (2.8). A single commodity flow formulation was introduced by Gavish and Graves in [28], and a two commodity flow formulation was presented by Finke, Claus, and Gun [29]. However, both of these formulations involve $(|V| - 1)$ as a coefficient. Therefore, the maximal absolute value of a subdeterminant grows at least linearly in $|V|$.

Claus [30] proposed the following multi-commodity flow formulation. Choose any $v^* \in V$ and set $V^* = V \setminus \{v^*\}$. The formulation then uses (2.7) and appends

$$\begin{aligned} y_{(v,w);k} - x_{\{v,w\}} &\leq 0 && \text{for all } \{v,w\} \in E \text{ and all } k \in V^*, \\ \sum_{w:\{v,w\}\in E} y_{(w,v);k} - y_{(v,w);k} &= c_{v;k} && \text{for all } v \in V \text{ and all } k \in V^*, \\ y_{(v,w);k} &\in \mathbb{Z}_{\geq 0} && \text{for all } \{v,w\} \in E \text{ and all } k \in V^*, \end{aligned}$$

where $c_{v^*;k} = -1, c_{k;k} = 1$ and $c_{v;k} = 0$ otherwise. The additional variables $y_{(v,w);k}$ represent flow of a commodity k along the arc (v, w). The constraints ensure that for every vertex $v \neq v^*$, one unit of flow of commodity v can be sent from v^* to v.

To construct a submatrix of the constraint matrix which has a large determinant, partition V^* into two sets $S, V^* \setminus S$ such that $|S| = \lfloor |V^*|/2 \rfloor$. For all $v \in S$ let $w(v), u(v) \in V$ be two vertices such that $\{v, w(v)\}, \{v, u(v)\} \in E$ are two distinct edges incident to v (if it is not possible to choose such vertices, the instance is trivially infeasible). Furthermore, let $k(v) \in V^* \setminus S$ be a distinct vertex for all $v \in S$, i. e. $k(v) \neq k(w)$ for all $v, w \in S$ with $v \neq w$. We then consider the submatrix A obtained by selecting the columns corresponding to the variables

$$x_{\{v,w(v)\}}, \quad x_{\{v,u(v)\}}, \quad y_{(w(v),v);v}, \quad y_{(u(v),v);v}, \quad y_{(v,w(v));k(v)}, \quad y_{(u(v),v);k(v)}$$

and the rows corresponding to the restrictions

$$y_{(s,t);k} - x_{\{s,t\}} \leq 0 \quad \text{for } (s,t,k) \in \{(v,w(v),v), (v,u(v),v), (v,w(v),k(v)), (u(v),v,k(v))\}$$

and

$$\sum_{s:\{t,s\}\in E} y_{(s,t);k} - y_{(t,s);k} = c_{t;k} \quad \text{for } (t,k) \in \{(v,v), (v,k(v))\}$$

for all $v \in S$. The submatrix $\tilde{A}$ of A corresponding to a fixed $v \in S$ is

$$\tilde{A} = \begin{pmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 \end{pmatrix}$$

with $\det(\tilde{A}) = 2$. By the described construction, A is a block diagonal matrix with $\lfloor (|V| - 1)/2 \rfloor$ copies of $\tilde{A}$ on the diagonal, implying $\det(A) = 2^{\lfloor (|V|-1)/2 \rfloor}$ and hence yielding an exponential lower bound on the largest absolute value of a subdeterminant of the constraint matrix.

To find the submatrix A, in the first step, a computer search for subdeterminants of absolute value strictly larger than 1 in small instances was employed. Based on the results, this general submatrix was derived. In this computer search, the unimodularity test implemented by Walter and Truemper [31] was of great help.

2.6 Steiner tree

The Steiner tree problem is another graph-theoretic problem found on Karp's list. Like the Hamiltonian cycle problem, the Steiner tree problem asks for a subgraph of a given graph with certain properties.

Problem 12 (Steiner tree). *Given a graph $G = (V, E)$, a subset of vertices $S \subseteq V$, a weight function $w : E \to \mathbb{Z}_{>0}$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether G has a subtree T of weight at most k, i. e. $\sum_{e \in E(T)} w(e) \leq k$, and such that $S \subseteq V(T)$.*

Let H be a connected subgraph of G that contains all vertices in S and has weight at most k. Then any spanning tree of H is a solution to the Steiner tree problem. In particular, the Steiner tree problem is equivalent to deciding whether such a connected subgraph exists. Furthermore, observe that in the integer programming formulations of the Hamiltonian cycle problem, the constraints added to (2.7) only need to ensure the subgraph corresponding to a solution is connected. In particular, the subtour elimination constraints presented in Section 2.5.1 and in Section 2.5.3 can be adapted to yield formulations of the Steiner tree problem. A detailed description of these formulations can be found in [32]. The analysis of subdeterminants can be conducted analogously to the one presented in Section 2.5.1 and Section 2.5.3 and yields unbounded subdeterminants for these formulations of the Steiner tree problem as well.

2.7 Max cut

The final graph theoretic problem we consider is the max cut problem.

Problem 13 (Max cut). *Given a graph $G = (V, E)$, a weight function $w : E \to \mathbb{Z}_{>0}$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a set $S \subseteq V$ of vertices such that $\sum_{\{u,v\} \in E:\, u \in S,\, v \notin S} w(\{u, v\}) \geq k$.*

Setting $\tilde{E} = V \times V$ and writing $\tilde{w}(e) = w(e)$ for all $e \in E$ and $\tilde{w}(e) = 0$ for all $e \in \tilde{E} \setminus E$, Poljak and Tuza [33] presented the following integer programming formulation of the problem

$$\begin{aligned} \sum_{e \in \tilde{E}} x_e \tilde{w}(e) &\geq k, & & \\ x_{\{u,v\}} + x_{\{u,w\}} + x_{\{v,w\}} &\leq 2 & &\text{for all } u, v, w \in V, |\{u, v, w\}| = 3, \\ x_{\{u,v\}} - x_{\{u,w\}} - x_{\{v,w\}} &\leq 0 & &\text{for all } u, v, w \in V, |\{u, v, w\}| = 3, \\ x_e &\in \{0, 1\} & &\text{for all } e \in \tilde{E}, \end{aligned} \tag{2.12}$$

Noticing the similarity of (2.12) and (2.6), we immediately see that the maximal absolute value subdeterminant of the constraint matrix of this integer program formulation grows at least exponentially in $|V|$.

Another formulation that uses variables for all edges, as well as all vertices, is given by

$$\begin{aligned} &\sum_{e \in E} x_e w(e) \geq k, \\ &\quad x_{\{u,v\}} \geq y_u - y_v && \text{for all } \{u,v\} \in E, \\ &\quad x_{\{u,v\}} \leq y_u + y_v && \text{for all } \{u,v\} \in E, \\ &\quad x_{\{u,v\}} \leq 2 - (y_u + y_v) && \text{for all } \{u,v\} \in E, \end{aligned} \tag{2.13}$$

and was presented in a more general setting by Chopra and Rao [34]. In this formulation, a vertex $v \in V$ is in the set S corresponding to a solution if and only if $y_v = 1$. The constraints ensure that only variables x_e are set to 1 in a solution if the edge e crosses from S to $V \setminus S$. Since the incidence matrix of G appears as a submatrix of the constraint matrix of this formulation, Lemma 1.3 implies that there are instances with exponentially (in $|V|$) large absolute values of subdeterminants.

2.8 Set packing and set covering

We now move on to set and partition problems. To begin, we consider two closely related problems that each ask to find a family of subsets within a given set of subsets that satisfy certain properties. Formally, the two problems can be defined in the following way.

Problem 14 (Set packing)**.** *Given a finite set $\mathcal{S}$, a finite family of subsets $S_1, \ldots, S_n \subseteq \mathcal{S}$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a subset $I \subseteq [n]$ of size at least k such that $S_i \cap S_j = \emptyset$ for all $i, j \in I$ with $i \neq j$.*

Problem 15 (Set covering)**.** *Given a finite set $\mathcal{S}$, a finite family of subsets $S_1, \ldots, S_n \subseteq \mathcal{S}$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a subset $I \subseteq [n]$of size at most k such that $\bigcup_{i \in I} S_i = \mathcal{S}$.*

The two problems can be formulated as the nearly identical integer programs

$$\begin{aligned} &\sum_{i \in [n]} x_i \geq k, \\ &\sum_{i \in [n] : s \in S_i} x_i \leq 1 && \text{for all } s \in \mathcal{S}, \\ &\quad x_i \in \{0,1\} && \text{for all } i \in [n] \end{aligned} \tag{2.14}$$

for the set packing problem, and

$$\begin{aligned} &\sum_{i \in [n]} x_i \leq k, \\ &\sum_{i \in [n] : s \in S_i} x_i \geq 1 && \text{for all } s \in \mathcal{S}, \\ &\quad x_i \in \{0,1\} && \text{for all } i \in [n] \end{aligned}$$

for the set covering problem.

To see that there are instances that contain subdeterminants with arbitrarily large absolute values, we consider the reduction from the clique problem to the set packing problem presented by Karp. Given an instance of the clique problem, i. e. a graph $G = (V, E)$ and an integer $l \in \mathbb{Z}_{>0}$, define an instance of the set packing problem by setting $\mathcal{S} = (V \times V) \setminus E$, $S_v = \{\{v, w\} : v, w \in V, \{v, w\} \notin E\}$ for all $v \in V$ and $k = l$. It is easy to see that in this case, the constraint matrix of the set of restrictions (2.14) is the incidence matrix of $\overline{G}$. Due to Lemma 1.3, this means unbounded subdeterminants are present in this formulation. Analogously, a reduction of the vertex cover problem to the set covering problem yields instances of the set covering problem where subdeterminants with arbitrarily large absolute values are present.

2.9 Exact cover and hitting set

The exact cover problem is similar to the set covering problem discussed in the previous section but asks for a disjoint union.

Problem 16 (Exact cover). *Given a finite set $\mathcal{S}$ and a finite family of subsets $S_1, \dots, S_n \subseteq \mathcal{S}$, decide whether there is a subset $I \subset [n]$ such that $\sqcup_{i \in I} S_i = \mathcal{S}$, where $\sqcup$ denotes the disjoint union.*

Setting $A = (a_{s,i}) \in \{0,1\}^{\mathcal{S} \times [n]}$, where $a_{s,i} = 1$ if $s \in S_i$ and $a_{s,i} = 0$ otherwise, an integer programming formulation of the problem is

$$\begin{aligned} Ax &= \mathbf{1} \\ x &\in \{0,1\}^n. \end{aligned}$$

In other words, the problem asks to find a subset of columns of A such that in every row, there is exactly one 1. Similarly, one can ask to find a subset of rows of A such that in every column, there is exactly one 1. This problem is also found on Karp's list and is called the hitting set problem.

Problem 17 (Hitting Set). *Given a finite set $\mathcal{S}$ and a finite family of subsets $S_1, \dots, S_n \subseteq \mathcal{S}$, decide whether there is a subset $S^* \subseteq \mathcal{S}$ such that $|S^* \cap S_i| = 1$ for all $i \in [n]$.*

Reusing the matrix A defined above, an integer program formulation for the hitting set problem is given by

$$\begin{aligned} A^T x &= \mathbf{1} \\ x &\in \{0,1\}^n. \end{aligned}$$

Note that A can be any matrix with entries in $\{0,1\}$ with distinct columns and no all-zero rows. Clearly, there are matrices of this type that have exponential subdeterminants in n and $|\mathcal{S}|$.

2.10 3-dimensional matching

We close the discussion of set and partition problems by examining the 3-dimensional matching problem.

Problem 18 (3-dimensional matching). *Given $M \subseteq X \times Y \times Z$, where $|X| = |Y| = |Z| = n$, decide whether there is $M' \subseteq M$ with $|M'| = n$ such that for $(x,y,z), (x',y',z') \in M'$ either $(x,y,z) = (x',y',z')$ or $x \neq x', y \neq y', z \neq z'$.*

An integer programming formulation of the problem is

$$\begin{aligned} &\sum_{m \in M} w_m \geq n && \\ &\sum_{m \in M_x} w_m \leq 1 && \text{where } M_x = \{(x',y',z') \in M : x' = x\} \text{ for all } x \in X, \\ &\sum_{m \in M_y} w_m \leq 1 && \text{where } M_y = \{(x',y',z') \in M : y' = y\} \text{ for all } y \in Y, \\ &\sum_{m \in M_z} w_m \leq 1 && \text{where } M_z = \{(x',y',z') \in M : z' = z\} \text{ for all } z \in Z. \end{aligned}$$

To construct instances with large subdeterminants, let $k \in \mathbb{N}$ and set $n = 2k$,

$$X = \bigcup_{i \in [k]} \{x_{i,1}, x_{i,2}\}, \quad Y = \bigcup_{i \in [k]} \{y_{i,1}, y_{i,2}\}, \quad Z = \bigcup_{i \in [k]} \{z_{i,1}, z_{i,2}\}.$$

and

$$M = \bigcup_{i\in[k]} \{(x_{i,1}, y_{i,2}, z_{i,2}), (x_{i,2}, y_{i,1}, z_{i,2}), (x_{i,2}, y_{i,2}, z_{i,1})\}.$$

Note that the constraint matrix of the system above has one column for every $m \in M$ and one row for every $x \in X, y \in Y$ and $z \in Z$. We now consider the submatrix A that is given by selecting the rows corresponding to the elements $x_{i,2}, y_{i,2}, z_{i,2}$ for all $i \in [k]$ and all columns. After reordering the rows and columns, A is a block diagonal matrix with k copies of the matrix $\tilde{A} = \left(\begin{smallmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{smallmatrix}\right)$ on the diagonal. Therefore $|\det(A)| = |\det(\tilde{A})^k| = 2^k = 2^{n/2}$. However, if $M' \subseteq M$ were a feasible solution at most one element of $\{(x_{i,1}, y_{i,2}, z_{i,2}), (x_{i,2}, y_{i,1}, z_{i,2}), (x_{i,2}, y_{i,2}, z_{i,1})\}$ could be in M' for any $i \in [k]$. This would imply $|M'| \leq k = n/2 < n = |M'|$, a contradiction, which means all these instances are infeasible. In particular, there is a trivial transformation to an equivalent instance with bounded subdeterminants (any fixed infeasible instance suffices).

Therefore, an interesting follow-up question is whether there is a subset of non-trivial instances where unbounded subdeterminants naturally appear. To answer this, we consider the reduction of 3-SAT, defined in Problem 3, to the 3-dimensional matching problem presented by Garey and Johnson [6]. For an instance of 3-SAT with literals $\{x_1, \ldots, x_n\} \cup \{\overline{x}_1, \ldots, \overline{x}_n\}$ and clauses $C = \{c_1, \ldots, c_m\}$, they define an instance of the 3-dimensional matching problem by setting

$$\begin{aligned}
X &= \{a_{i,j} : i \in [n], j \in [m]\} \cup \{s_{1,j} : j \in [m]\} \cup \{g_{1,k} : k \in [m(n-1)]\}, \\
Y &= \{b_{i,j} : i \in [n], j \in [m]\} \cup \{s_{2,j} : j \in [m]\} \cup \{g_{2,k} : k \in [m(n-1)]\}, \\
Z &= \{x_{i,j}, \overline{x}_{i,j} : i \in [n], j \in [m]\}, \\
\text{and} \quad M &= \left(\bigcup_{i=1}^{n} T_i\right) \cup \left(\bigcup_{j=1}^{m} C_j\right) \cup G, \\
\text{where} \quad T_i &= \{(\overline{x}_{i,j}, a_{i,j}, b_{i,j}) : j \in [m]\} \cup \{(x_{i,j}, a_{i,j+1}, b_{i,j}) : j \in [m-1]\} \cup \{(x_{i,m}, a_{i,1}, b_{i,m})\} \\
C_j &= \{(x_{i,j}, s_{1,j}, s_{2,j}) : x_i \in c_j\} \cup \{(\overline{x}_{i,j}, s_{1,j}, s_{2,j}) : \overline{x}_i \in c_j\} \\
G &= \{(x_{i,j}, g_{1,k}, g_{2,k}), (\overline{x}_{i,j}, g_{1,k}, g_{2,k}) : i \in [n], j \in [m], k \in [m(n-1)]\}.
\end{aligned}$$

For the intuition behind this reduction as well as a proof of the correctness, see [6]. We now consider the submatrix A of the constraint matrix of the instance of the 3-dimensional matching problem that stems from this reduction that is obtained by selecting the rows corresponding to the elements

$$a_{i,2}, \quad b_{i,2}, \quad x_{i,1}, \quad x_{i,2}, \quad g_{1,i}$$

and the columns corresponding to the triples

$$(\overline{x}_{i,2}, a_{i,2}, b_{i,2}), (x_{i,1}, a_{i,2}, b_{i,1}), (x_{i,2}, a_{i,3}, b_{i,2}), (x_{i,1}, g_{1,i}, g_{2,i}), (x_{i,2}, g_{1,i}, g_{2,i})$$

for all $i \in [n]$.

We see that for each fixed $i \in [n]$ the corresponding submatrix is

$$\tilde{A} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

with $\det(\tilde{A}) = 2$. Furthermore, A is a block diagonal matrix with n copies of $\tilde{A}$ on its diagonal, which implies $\det(A) = 2^n$. Since these instances stem from a reduction of 3-SAT to the 3-dimensional matching problem, the version of the 3-dimensional matching problem restricted to instances of this type remains NP-hard. Furthermore, there is no obvious way to transform these instances into equivalent ones that do not feature unbounded subdeterminants.

2.11 Weakly NP-complete problems

Since all problems on Karp's list are NP-complete, there are no known polynomial time algorithms where the polynomial is in the size of the instance. The key difference of the problems we will discuss in this section is that there are polynomial time algorithms in the size of the instance and in the numeric value of the given integers. These algorithms are called *pseudo-polynomial*, and an NP-complete problem that admits a pseudo-polynomial-time algorithm is called *weakly NP-complete*. [35]

2.11.1 Partition and subset sum

The first such problem we will discuss is the partition problem.

Problem 19 (Partition). *Given n integers $a_i, \dots, a_n \in \mathbb{Z}$ decide whether there is a partition $S, T \subseteq [n]$ of $[n]$, i.e. $S \sqcup T = [n]$, such that $\sum_{i \in S} a_i = \sum_{j \in T} a_j$.*

Actually, the partition problem is a special case of another problem on Karp's list, the subset sum problem.

Problem 20 (Subset sum). *Given n integers $a_i, \dots, a_n \in \mathbb{Z}$ and another integer $b \in \mathbb{Z}$, decide whether there is a subset $S \subseteq [n]$ such that $\sum_{i \in S} a_i = b$.*

In Karp's paper, this problem was listed under the name "Knapsack". However, this name is now usually used for a different problem (see, for example, [36]). For the subset sum problem, there is an algorithm that runs in time $\mathcal{O}(nb)$ [36, 37]. This implies that instances where b is polynomial in n can be solved in polynomial time. Note that it is easy to check $\max\{a_1, \dots, a_n\} \geq b/n$ in polynomial time and that any instance that violates this inequality is infeasible. This implies that any instance with $\max\{a_1, \dots, a_n\}$ polynomially bounded in n can be solved in polynomial time. Therefore, any integer programming formulation that has the a_i appear as coefficients has a constraint matrix with subdeterminants that are not polynomially bounded for hard instances.

So far, for all problems we have considered, there were natural integer programming formulations in which all entries of the constraint matrices were bounded by some constant independent of the instance at hand. Assuming that any natural integer programming formulation of the subset sum problem has the a_i as coefficients, this pattern is broken here. Note, however, that this crucially depends on the notion of a *natural formulation*. Since the previously discussed problems are NP-complete and admit integer programming formulations with bounded coefficients, there are integer programming formulations for any NP-complete problem with bounded coefficients.

2.11.2 Job sequencing

The final problem is set in the industrial world. The task is to schedule a set of jobs that have to be completed sequentially on a machine, where each job takes a certain time to complete and has a deadline associated with it. Additionally, a penalty is incurred for each missed deadline. The goal is to keep the total penalty below a threshold.

Problem 21 (Job sequencing). *Given an execution time vector $(t_1, \dots, t_n) \in \mathbb{Z}^n_{>0}$, a deadline vector $(d_1, \dots, d_n) \in \mathbb{Z}^n_{>0}$, a penalty vector $(p_1, \dots, p_n) \in \mathbb{Z}^n_{>0}$ and an integer $k \in \mathbb{Z}_{>0}$, decide whether there is a permutation π of $[n]$ such that the sum of penalties $\sum_{i=1}^n a_i p_{\pi(i)}$ is at most k where*

$$a_i = \begin{cases} 1, & \text{if } \sum_{j=1}^{i} t_{\pi(j)} > d_{\pi(i)} \\ 0, & \text{otherwise} \end{cases}$$

for all $i \in [n]$.

Let $M = \sum_{i=1}^{n} t_i$. A straightforward approach to come up with an integer programming formulation is to introduce a variable $x_i \in \{0, 1\}$ for each $i \in [n]$ that takes the value 1 if task i is not completed on time and 0 otherwise, and to add the restriction $\sum_{i=1}^{n} x_i p_i \leq k$. To set the variables x_i correctly, the so-called *big-M* or *indicator* constraint [38]

$$[\text{the time task } i \text{ finishes}] \leq d_i + Mx_i \tag{2.15}$$

can be used, where "[the time task i finishes]" is a suitable linear expression in some variables of the formulation. Baker and Keller [39] present six integer programming formulations for a similar problem, the first five of which can be easily adapted to the job sequencing problem using this idea. However, a pseudo-polynomial time algorithm that runs in $\mathcal{O}(nM)$ was found by Lawler and Moore [40], which implies that M must not be polynomially bounded in n in instances for which no polynomial-time algorithm is known. We say that M is *super-polynomial* in n in this case. Since M appears as a coefficient in (2.15), this formulation features unbounded subdeterminants.

The last formulation is obtained by adapting the *time-indexed model* proposed by Baker and Keller. The formulation uses the variables $x_{i,t} \in \{0, 1\}$ for $i \in [n], t \in [M - t_i + 1]$ that take the value 1 if task i is started at time t. This means task i is processed in the timesteps $t, \ldots, t + t_i - 1$. Furthermore, let

$$S_{i,t} = \{s \in [M - t_i + 1] : s \leq t \leq s + t_i - 1\}$$

denote the set of times when the task i can be started so that the time t falls within its execution. The formulation then introduces the restrictions

$$\begin{aligned}
\sum_{t=1}^{M-t_i+1} x_{i,t} &= 1 && \text{for all } i \in [n], \\
\sum_{i=1}^{n} \sum_{s \in S_{i,t}} x_{i,s} &\leq 1 && \text{for all } t \in [M], \\
\sum_{i=1}^{n} p_i \sum_{t=d_i-t_i+2}^{M-t_i+1} x_{i,t} &\leq k.
\end{aligned}$$

that ensure that every task is started at exactly one point in time, at most one task is processed at the same time, and that the sum of penalties does not exceed k.

Unlike the previously mentioned formulations, this formulation does not include M or any of the t_i as coefficients. Instead, it features a constraint matrix of size $((n-1)M+n) \times (M+n+1)$. Recalling that M has to be super-polynomial in n in instances for which no polynomial-time algorithm is known, we see that in this case, this integer programming formulation has a constraint matrix with super-polynomially many rows and columns. Therefore, even if the subdeterminants were bounded and there was a polynomial-time algorithm to solve the integer program, this would not yield a polynomial-time algorithm for the original problem since the integer program does not have polynomial size compared to the original instance.

In addition, we can actually find submatrices with large determinants for most instances. To this end, define $l_1 = 1$ and $l_i = l_{i-2} + t_{i-1} + 1$ for $i \in \{2k + 1 : k \in \mathbb{Z}_{>0}, 2k + 2 \in [n]\}$. By reordering and taking some $\tilde{n} \leq n$ we can assume $2 \leq t_1 \leq t_2 \leq \ldots \leq t_{\tilde{n}}$ and $l_i + t_i \leq M - t_i + 1$ as well as $l_i + 1 \leq M - t_{i+1} + 1$ for all $i \in [\tilde{n}]$. This ensures that all variables and restrictions we are about to select are actually part of the integer program. Now, consider the submatrix A that arises by selecting the columns corresponding to the variables

$$x_{i,l_i}, \quad x_{i,l_i+t_i}, \quad x_{i+1,l_i+1},$$

and the rows corresponding to the restrictions

$$\sum_{t=1}^{M-t_i+1} x_{i,t} = 1, \qquad \text{and} \qquad \sum_{j=1}^{n} \sum_{s \in S_{j,t}} x_{j,s} \leq 1 \quad \text{for } t \in \{l_i + t_i - 1, l_i + t_i\}$$

for all $i \in \{2k-1 : k \in \mathbb{Z}_{>0}, 2k \in [\tilde{n}]\}$. The submatrix $\tilde{A}$ of A that corresponds to a fixed i is

$$\tilde{A} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix},$$

where we utilized $2 \leq t_{i+1}$ to obtain the last column. Since we assumed $t_i \leq t_{i+1}$ and by the choice of the l_i, we see that A forms a upper block triangular matrix with $\lfloor \tilde{n}/2 \rfloor$ copies of $\tilde{A}$ on the diagonal. This means $|\det(A)| = |\det(\tilde{A})^{\lfloor \tilde{n}/2 \rfloor}| = 2^{\lfloor \tilde{n}/2 \rfloor}$. Furthermore, it is not difficult to see that there are many instances for which this submatrix yields an exponential lower bound in n. In particular, this is the case for all instances where $\tilde{n}$ can be chosen to be at least $n/2$.

3 Conclusion

We examined various integer programming formulations for NP-complete problems and were able to show that all of them have unbounded subdeterminants. Clearly, there is a vast amount of further integer programming formulations for the problems on Karp's list that we did not consider (and especially so for other problems not on the list). We want to emphasize that we did not include these formulations not because we tried to prove the presence of unbounded subdeterminants and failed but because we simply did not examine them. Given the fact that the presented strategies could be applied successfully to all integer programming formulations that we did examine, we firmly believe that this is also true for a large number of formulations we did not consider.

However, this obviously does not prove that any formulation of an NP-hard problem must necessarily feature unbounded subdeterminants. Rather, the situation is similar to why we believe P to be unequal to NP: Many people have looked at a large number of algorithms for many different NP-complete problems, and none of them had a polynomial runtime. Similarly, people have come up with a large number of integer programming formulations for many NP-complete problems, and we know of none which has bounded subdeterminants. However, unlike in the case of polynomial time algorithms for NP-complete problems, to our knowledge, nobody has actively tried to come up with an integer programming formulation for an NP-complete problem that has bounded subdeterminants by design. Therefore, an interesting approach for future research is to attempt the construction of such an integer programming formulation.

Lastly, observe that the max cut problem is trivial for bipartite graphs, in which case the formulation (2.13) has subdeterminants bounded by 1. An open question is whether this result can be generalized to instances whose corresponding integer programs have subdeterminants bounded by any constant. More generally, one could choose any specific problem and corresponding integer programming formulation and try to find a polynomial time algorithm for all instances whose corresponding integer programs have maximal subdeterminants bounded by a constant. This could provide valuable additional insights into the relation of subdeterminants of an integer programming formulation and the difficulty of the underlying problem.

Bibliography

[1] S. I. Veselov and A. J. Chirkov. "Integer Program with Bimodular Matrix". In: *Discrete Optimization* 6.2 (2009), pp. 220–222.

[2] S. Artmann, R. Weismantel, and R. Zenklusen. "A Strongly Polynomial Algorithm for Bimodular Integer Linear Programming". In: *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*. STOC 2017. Montreal, Canada: Association for Computing Machinery, 2017, pp. 1206–1219.

[3] S. Fiorini, G. Joret, S. Weltge, and Y. Yuditsky. "Integer programs with bounded subdeterminants and two nonzeros per row". In: *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*. Denver, CO, USA, 2022, pp. 13–24.

[4] M. Nägele, C. Nöbel, R. Santiago, and R. Zenklusen. "Advances on Strictly Δ-Modular IPs". In: *Proceedings of the 24th Conference on Integer Programming and Combinatorial Optimization (IPCO '23)*. 2023, pp. 393–407.

[5] R. M. Karp. "Reducibility Among Combinatorial Problems." In: *Complexity of Computer Computations*. Ed. by R. E. Miller and J. W. Thatcher. The IBM Research Symposia Series. Plenum Press, New York, 1972, pp. 85–103.

[6] M. R. Garey and D. S. Johnson. *Computers and Intractability*. WH Freeman & Co, 1979.

[7] L. G. Khachiyan. "A Polynomial Algorithm in Linear Programming". In: *Doklady Akademii Nauk SSSR* 244.5 (1979), pp. 1093–1096. [English translation: Soviet Mathematics Doklady 20.1 (1979), pp. 191-194].

[8] S. A. Cook. "The Complexity of Theorem-Proving Procedures". In: *Proceedings of the Third Annual ACM Symposium on Theory of Computing*. STOC '71. Shaker Heights, Ohio, USA: Association for Computing Machinery, 1971, pp. 151–158.

[9] S. R. Chestnut and R. Zenklusen. "Interdicting Structured Combinatorial Optimization Problems with $\{0, 1\}$-Objectives". In: *Math. Oper. Res.* 42.1 (Jan. 2017), pp. 144–166.

[10] J. W. Grossman, D. M. Kulkarni, and I. E. Schochetman. "On the minors of an incidence matrix and its Smith normal form". In: *Linear Algebra and its Applications* 218 (1995), pp. 213–224.

[11] J. Lee, J. Paat, I. Stallknecht, and L. Xu. "Polynomial Upper Bounds on the Number of Differing Columns of Δ-Modular Integer Programs". In: *Mathematics of Operations Research* (2022).

[12] E. Mendelson. *Introduction to Mathematical Logic*. 4th ed. Chapman & Hall, London, 1997.

[13] B. Aspvall, M. F. Plass, and R. E. Tarjan. "A linear-time algorithm for testing the truth of certain quantified boolean formulas". In: *Information Processing Letters* 8.3 (1979), pp. 121–123.

[14] D. Kardos, P. Patassy, S. Szabó, and B. Zaválnij. "Numerical experiments with LP formulations of the maximum clique problem". In: *Central European Journal of Operations Research* 30.4 (Dec. 2022), pp. 1353–1367.

[15] J. Moon and L. Moser. "On cliques in graphs". In: *Israel Journal of Mathematics* 3.1 (1965), pp. 23–28.

[16] I. Méndez-Díaz and P. Zabala. "A Branch-and-Cut algorithm for graph coloring". In: *Discrete Applied Mathematics* 154.5 (2006). IV ALIO/EURO Workshop on Applied Combinatorial Optimization, pp. 826–847.

[17] A. Mehrotra and M. A. Trick. "A Column Generation Approach for Graph Coloring". In: *INFORMS Journal on Computing* 8.4 (1996), pp. 344–354.

[18] M. Campêlo, R. Corrêa, and Y. Frota. “Cliques, holes and the vertex coloring polytope”. In: *Information Processing Letters* 89.4 (2004), pp. 159–164.

[19] M. Campêlo, V. A. Campos, and R. C. Corrêa. “On the asymmetric representatives formulation for the vertex coloring problem”. In: *Discrete Applied Mathematics* 156.7 (2008). GRACO 2005, pp. 1097–1111.

[20] A. Jabrayilov and P. Mutzel. “New Integer Linear Programming Models for the Vertex Coloring Problem”. In: *LATIN 2018: Theoretical Informatics*. Ed. by M. A. Bender, M. Farach-Colton, and M. A. Mosteiro. Cham: Springer International Publishing, 2018, pp. 640–652.

[21] A. Jabrayilov and P. Mutzel. “Strengthened Partial-Ordering Based ILP Models for the Vertex Coloring Problem”. In: *ArXiv* abs/2206.13678 (2022).

[22] F. A. Chudak, M. X. Goemans, D. S. Hochbaum, and D. P. Williamson. “A primal–dual interpretation of two 2-approximation algorithms for the feedback vertex set problem in undirected graphs”. In: *Operations Research Letters* 22.4 (1998), pp. 111–118.

[23] A. Baharev, H. Schichl, A. Neumaier, and T. Achterberg. “An Exact Method for the Minimum Feedback Arc Set Problem”. In: *ACM J. Exp. Algorithmics* 26 (Apr. 2021).

[24] M. Grötschel, M. Jünger, and G. Reinelt. “A Cutting Plane Algorithm for the Linear Ordering Problem”. In: *Operations Research* 32.6 (1984), pp. 1195–1220.

[25] G. Dantzig, R. Fulkerson, and S. Johnson. “Solution of a Large-Scale Traveling-Salesman Problem”. In: *Journal of the Operations Research Society of America* 2.4 (1954), pp. 393–410.

[26] T. Akiyama, T. Nishizeki, and N. Saito. “NP-Completeness of the Hamiltonian Cycle Problem for Bipartite Graphs”. In: *Journal of Information Processing* 3 (1980), pp. 73–76.

[27] C. E. Miller, A. W. Tucker, and R. A. Zemlin. “Integer Programming Formulation of Traveling Salesman Problems”. In: *J. ACM* 7.4 (1960), pp. 326–329.

[28] B. Gavish and S. C. Graves. “The Travelling Salesman Problem and Related Problems”. In: Operations Research Center Working Paper (1978). Massachusetts Institute of Technology, Operations Research Center, OR 078-78.

[29] G. Finke, A. Claus, and E. Gun. “A Two-commodity Network Flow approach to the Traveling Salesman Problem”. In: *Congressus numerantium* 41 (1984), pp. 167–178.

[30] A. Claus. “A New Formulation for the Travelling Salesman Problem”. In: *SIAM Journal on Algebraic Discrete Methods* 5.1 (1984), pp. 21–25.

[31] M. Walter and K. Truemper. “Implementation of a unimodularity test”. In: *Mathematical Programming Computation* 5.1 (2013), pp. 57–73.

[32] M. X. Goemans and Y.-S. Myung. “A catalog of steiner tree formulations”. In: *Networks* 23.1 (1993), pp. 19–28.

[33] S. Poljak and Z. Tuza. “The expected relative error of the polyhedral approximation of the max-cut problem”. In: *Operations Research Letters* 16.4 (1994), pp. 191–198.

[34] S. Chopra and M. R. Rao. “The partition problem”. In: *Mathematical Programming* 59 (Mar. 1993), pp. 87–115.

[35] L. Hall. “Computational Complexity”. In: *Encyclopedia of Operations Research and Management Science*. Ed. by S. I. Gass and M. C. Fu. Boston, MA: Springer US, 2013, pp. 238–241.

[36] A. Polak, L. Rohwedder, and K. Węgrzycki. “Knapsack and Subset Sum with Small Items”. In: *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*. Ed. by N. Bansal, E. Merelli, and J. Worrell. Vol. 198. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 106:1–106:19.

[37] R. Bellman. *Dynamic Programming*. 1st ed. Princeton, NJ, USA: Princeton University Press, 1957.

[38] P. Belotti et al. “On handling indicator constraints in mixed integer programming”. In: *Computational Optimization and Applications* 65.3 (Dec. 2016), pp. 545–566.

[39] K. R. Baker and B. Keller. “Solving the single-machine sequencing problem using integer programming”. In: *Computers & Industrial Engineering* 59.4 (2010), pp. 730–735.

[40] E. L. Lawler and J. M. Moore. “A Functional Equation and Its Application to Resource Allocation and Sequencing Problems”. In: *Management Science* 16.1 (1969), pp. 77–84.